

JavaScript Topics and Questions

Q1. Explain var, let, and const differences with an example.

Problem: Why do we use let and const instead of var? Show with code.

Answer:

```
// var is function-scoped, allows redeclaration
var x = 10;
var x = 20;
console.log(x); // 20

// let is block-scoped, no redeclaration
let y = 10;
// let y = 20; // Error
y = 20;
console.log(y); // 20

// const is block-scoped, cannot be reassigned
const z = 10;
// z = 20; // Error
console.log(z); // 10
```

Key Point:

- var: function scoped, hoisted with undefined.
- let: block scoped, hoisted but in **Temporal Dead Zone** until initialized.
- const: block scoped, must be initialized, cannot be reassigned.

Video:

Variables - <https://www.scaler.com/topics/course/javascript-beginners/video/243/>

Hoisting- <https://www.scaler.com/topics/course/javascript-beginners/video/255/>

Q2. What is hoisting in JavaScript?

Problem: Explain hoisting with a function and variable example.

Solution:

```
// Function declarations are hoisted completely
sayHello();
function sayHello() {
  console.log("Hello");
}

// Variables with var are hoisted but initialized with undefined
console.log(a); // undefined
var a = 5;

// Variables with let/const are hoisted but not initialized
// console.log(b); // ReferenceError
let b = 10;
```

Key Point:

- Hoisting = moving declarations to the top of scope during compilation.
- Function declarations hoist fully.
- var hoists with undefined.
- let and const hoist but are not accessible until initialization (TDZ).

Q3. What are primitive vs reference types in JavaScript?

Problem: Explain the difference between primitive and reference types with an example.

Solution:

```
// Primitive types: stored by value
```

```
let a = 10;

let b = a;

b = 20;

console.log(a); // 10 (unchanged)

console.log(b); // 20

// Reference types: stored by reference

let obj1 = { name: "John" };

let obj2 = obj1;

obj2.name = "Doe";

console.log(obj1.name); // "Doe" (changed because same reference)
```

Key Point:

- **Primitives:** string, number, boolean, null, undefined, symbol, bigint → copied by value.
- **References:** object, array, function → copied by reference (both variables point to same memory).

Videos:

Primitives - <https://www.scaler.com/topics/course/javascript-beginners/video/298/>

Reference - <https://www.scaler.com/topics/course/javascript-beginners/video/299/>

Q4. Explain == vs === in JavaScript.

Problem: Why should we prefer === over ==? Show with examples.

Solution:

```
console.log(2 == "2");    // true  (type coercion happens)

console.log(2 === "2");  // false (strict equality)


console.log(null == undefined); // true

console.log(null === undefined); // false
```

Key Point:

- == → checks **value only**, allows type coercion.
- === → checks **value + type** (strict equality).
- Best practice: always use === unless there's a strong reason for loose equality.

Q5. What is scope in JavaScript?

Problem: Explain global, function, and block scope with examples.

Solution:

```
// Global Scope

var a = 10;

function test() {

  console.log(a); // 10 (accessible)

}


// Function Scope
```

```
function fn() {

  var b = 20;

  console.log(b);

}

// console.log(b); // Error

// Block Scope

if (true) {

  let c = 30;

  const d = 40;

}

// console.log(c, d); // Error (not accessible outside block)
```

Key Point:

- **Global scope:** accessible everywhere.
- **Function scope:** created by var and functions.
- **Block scope:** created by {} using let and const.

Q6. What are closures in JavaScript?

Problem: Explain closure with a real example.

```
function outer() {
  let count = 0;
  return function inner() {
    count++;
  }
}
```

```
    return count;
  };
}

const counter = outer();
console.log(counter()); // 1
console.log(counter()); // 2
```

Key Point :

- A closure is created when an **inner function remembers variables from its outer function's scope** even after the outer function has finished executing.
- Commonly used in:
 - **Data hiding / encapsulation**
 - **Function factories**
 - **Callbacks & event handlers**

Videos:

Closures - <https://www.scaler.com/topics/course/javascript-beginners/video/302/>

Closures with nested functions:

<https://www.scaler.com/topics/course/javascript-beginners/video/303/>

Q7. Implement a counter using closures

Problem Statement:

Suppose you are asked to design a counter function. Each time the counter is called, it should return the next number in sequence. For example, the first call should return **1**, the second call should return **2**, and so on.

The requirement is to implement this without using any global variables — the counter's state should be private and maintained internally.

Approach:

- Use a closure to create a private variable (count).
- Return an inner function that increments and returns the count each time it is called.
- Each counter created from the function should have its own independent state.

```
function createCounter() {  
  
  let count = 0; // private variable  
  
  return function() {  
  
    count++;  
  
    return count;  
  
  };  
  
}  
  
const counter1 = createCounter();  
  
console.log(counter1()); // 1  
  
console.log(counter1()); // 2  
  
console.log(counter1()); // 3
```

Output / Explanation:

- Each call increments count.
- The inner function keeps reference to count (closure).
- If you create a new counter (createCounter () again), it starts fresh.

Q8. Implement currying in JavaScript

Problem Statement:

Suppose you are asked to create a function `add` that can take its arguments **one at a time** instead of all at once. For example:

`add(2)(3)(4) // should return 9`

The requirement is to implement currying: transform a function with multiple arguments into a series of functions that each take a single argument.

Approach:

- Normally, a function `add(a, b, c)` takes three arguments at once.
- With currying, we nest functions so each one takes one argument and remembers it using closure.
- The innermost function finally computes the result
- Useful for partial application, reusability, and functional programming.

```
function add(a) {  
  
  return function(b) {  
  
    return function(c) {  
  
      return a + b + c;  
  
    };  
  
  };  
  
}  
  
console.log(add(2)(3)(4)); // 9
```


Output / Explanation:

- Each function remembers its argument via closure.
- Final call returns sum of all.
- In interviews, you may be asked to make it **generic** (handle any number of arguments).

Q9. Implement a once function in JavaScript

Problem Statement:

Imagine you are initializing a payment gateway in your application. You only want the initialization function to run **once**, even if the user tries to trigger it multiple times (e.g., clicking the “Pay Now” button several times).

The requirement is to design a once utility that ensures a given function can only run the first time it is called. On all subsequent calls, it should return the result of the first execution without running again.

Approach:

- Use a **closure** to keep track of whether the function has been called already.
- Store the result of the first execution.
- If called again → return the stored result instead of executing the function again.

Code:

```
function once(fn) {  
  
  let called = false;  
  
  let result;
```

```

return function(...args) {

  if (!called) {

    result = fn.apply(this, args); // run only first time

    called = true;

  }

  return result;

};

}

// Example usage

const initializePayment = once((amount) => {

  console.log("Payment gateway initialized");

  return `Processed payment of ${amount}`;

});

console.log(initializePayment(100)); // "Payment gateway initialized" + "Processed payment of $100"

console.log(initializePayment(200)); // Returns "Processed payment of $100" (ignores new call)

```

Output / Explanation:

- First call executes the original function (initializePayment) and stores result.
- Second call does not execute it again; it simply returns the stored result.

- Prevents duplicate operations like **API initialization, event binding, DB connection**, or costly computations.
-

Q10. Implement memoization for a function (e.g., factorial).

Problem Statement:

Suppose you are asked to write a program that calculates the factorial of a number. Factorial grows quickly and involves repeated recursive calculations. For example, to compute `factorial(5)`, you also compute `factorial(4)`, `factorial(3)`, etc. If later you call `factorial(5)` again, the same steps are repeated, wasting time.

How can you design a **memoization utility** so that the function “remembers” results of previous calls and reuses them instead of recalculating? Show how it would work with a factorial function.

Approach:

- Maintain a **cache object** inside a closure.
- When the function is called, first check if the result exists in cache.
- If yes → return cached result.
- If not → compute, store in cache, and return result.

Code:

```
function memoize(fn) {  
  
  const cache = {};  
  
  return function(...args) {  
  
    const key = JSON.stringify(args); // serialize arguments  
  
    if (cache[key]) {  
  
      console.log("Fetching from cache");  
  
      return cache[key];  
    }  
  }  
}
```

```

    }

    console.log("Computing result");

    const result = fn.apply(this, args);

    cache[key] = result;

    return result;
};
}

// Example: factorial

function factorial(n) {

    if (n <= 1) return 1;

    return n * factorial(n - 1);

}

const memoFactorial = memoize(factorial);

console.log(memoFactorial(5)); // Computing result -> 120

console.log(memoFactorial(5)); // Fetching from cache -> 120

```

Output / Explanation:

- First call to `memoFactorial(5)` → calculates recursively, stores result.
- Second call to `memoFactorial(5)` → instantly returns stored result, no recursion.

- Time-saving technique especially useful in recursive algorithms, API calls, and heavy computations.

Q11. Explain the JavaScript Event Loop.

Problem Statement:

In an interview, you might be asked: *"How does JavaScript handle asynchronous operations like `setTimeout` or Promises if it is single-threaded?"*

Solution (Conceptual):

- JavaScript has a **single-threaded execution model** → only one thing runs at a time.
- Asynchronous tasks (timers, network calls, promises) are handled using the **event loop**.
- Event loop checks two queues:
 - **Callback Queue (macrotasks)** → e.g., `setTimeout`, `setInterval`, DOM events.
 - **Microtask Queue** → e.g., `Promise.then`, `queueMicrotask`.
- **Order of execution:** Call stack → microtasks → macrotasks.

```
console.log("Start");

setTimeout(() => console.log("Timeout"), 0);
Promise.resolve().then(() => console.log("Promise"));

console.log("End");
```

Output:

Start

End

Promise

Timeout

Key Point:

- Promises (microtasks) run before timers (macrotasks).

Videos:

Event loop - <https://www.scaler.com/topics/course/javascript-beginners/video/289/>

setTimeout and setInterval -

<https://www.scaler.com/topics/course/javascript-beginners/video/291/>

Q12. Implement a sleep function using Promises.

Problem Statement:

Suppose you are asked to implement a sleep function that delays execution for a given number of milliseconds before resuming. Example:

```
sleep(2000).then(() => console.log("Executed after 2 seconds"));
```

Approach:

- From the usage, it is clear that we need to return a new Promise (because it is followed by then in the question)
- Use a Promise that resolves after setTimeout.
- The caller can use .then or await to wait for it.

Code:

```
function sleep(ms) {  
  return new Promise((resolve) => setTimeout(resolve, ms));  
}  
  
// Example usage
```

```
sleep(2000).then(() => console.log("Executed after 2 seconds"));

(async function test() {
  console.log("Before sleep");
  await sleep(1000);
  console.log("After 1 second");
})();
```

Explanation:

1. The sleep function itself

```
function sleep(ms) {
  return new Promise((resolve) => setTimeout(resolve, ms));
}
```

- sleep takes ms (milliseconds) as input.
- Inside, we return a **new Promise**.
- A Promise takes a function with resolve (and optionally reject).
- We use setTimeout(resolve, ms) → which means:
 - After ms milliseconds, resolve() is called.
 - This signals that the Promise is “done” and can continue.

So sleep(2000) → returns a Promise that resolves after 2000 ms.

2. Using .then

```
sleep(2000).then(() => console.log("Executed after 2 seconds"));
```

- Here, we call sleep(2000).
- That returns a Promise.
- We attach .then(() => ...) → which means - run this function once the Promise resolves.

- After 2 seconds → "Executed after 2 seconds" is logged.

Output:

Before sleep

(wait 1 second)

After 1 second

Key Takeaway:

- sleep doesn't block the thread (JS never blocks). Instead, it uses a Promise and resumes execution once the timer resolves.
- This is the JS equivalent of "pause execution for X ms" – useful in async code, retries, animations, or pacing API calls.

Q13. Difference between callbacks, promises, and async/await

Problem Statement:

In interviews, you might be asked: *Explain how callbacks, promises, and async/await differ in handling asynchronous code.*

Solution:

1. Callback

- Pass a function into another function to be executed later.
- Can cause **callback hell** if deeply nested.

```
function fetchData(callback) {  
  
  setTimeout(() => {  
  
    callback("Data received");  
  
  }, 1000);  
  
}
```



```
fetchData((data) => console.log(data)); // "Data received"
```

2. Promise

- Represents a value that may be available now, later, or never.
- Cleaner chaining with `.then()`, `.catch()`.

```
const fetchPromise = new Promise((resolve) =>

  setTimeout(() => resolve("Data received"), 1000)

);

fetchPromise.then((data) => console.log(data));
```

3. Async/Await

- Built on top of Promises.
- Looks synchronous, avoids chaining.

```
async function fetchData() {

  const data = await new Promise((resolve) =>

    setTimeout(() => resolve("Data received"), 1000)

  );

  console.log(data);

}

fetchData();
```

Key Point:

- **Callbacks** → low-level, messy in nested flows.
- **Promises** → more structured, chaining possible.
- **Async/Await** → syntactic sugar over promises, most readable.

Videos:

Promises- <https://www.scaler.com/topics/course/javascript-beginners/video/292/>

Async / Await - <https://www.scaler.com/topics/course/javascript-beginners/video/293/>

Q14. Implement a polyfill for `Promise.all`

Problem Statement:

Suppose you are asked to implement your own version of `Promise.all`. The requirements are:

1. Input: an array of Promises (or values).
2. Output: a new Promise that:
 - **resolves** when all Promises resolve → with results in the same order.
 - **rejects immediately** if any Promise rejects.

Example:

- `Promise.all([p1, p2, p3]) // → resolves [10, 20, 30]`

Intuition (what do we need?):

1. We need to **wait for multiple async tasks** to finish.

- This means: track how many are done.
- 2. We need to **preserve order**.
 - So even if p2 finishes before p1, its result should be stored in the right slot.
- 3. We need to **resolve once all are done**.
 - That means: compare a completed counter to total promises.
- 4. We need to **reject fast** if any fails.
 - As soon as one rejects, no point waiting → reject the outer promise immediately.
- 5. We need to handle **non-Promise values** too.
 - `Promise.all([1, Promise.resolve(2)])` → should work.
 - So wrap each input with `Promise.resolve(p)`.

Step-by-Step Plan:

- Create an outer Promise → `new Promise((resolve, reject) => { ... })`.
- Inside:
 - Initialize `results = []` and `completed = 0`.
 - Loop through input array:
 - Wrap each element with `Promise.resolve(p)` to normalize.
 - On resolve:
 - Store result in `results[index]`.
 - Increment `completed`.
 - If `completed === total`, call `resolve(results)`.
 - On reject: immediately call `reject(err)`

```
function promiseAll(promises) {  
  return new Promise((resolve, reject) => {  
    let results = [];  
    let completed = 0;  
  
    // Edge case: empty input  
    if (promises.length === 0) {
```

```

    return resolve([]);
  }

  promises.forEach((p, index) => {
    // Ensure we're working with a Promise
    Promise.resolve(p)
      .then((value) => {
        results[index] = value; // store in correct position
        completed++;

        // If all promises have resolved
        if (completed === promises.length) {
          resolve(results);
        }
      })
      .catch(reject); // reject immediately on first error
  });
});
}

// Example usage
const p1 = Promise.resolve(10);
const p2 = new Promise((res) => setTimeout(() => res(20), 500));
const p3 = Promise.resolve(30);

promiseAll([p1, p2, p3]).then(console.log);
// [10, 20, 30] after ~500ms

```

Expected Behavior:

- Resolves with array of values when all succeed.
- Rejects immediately if any promise fails.

Q15. Implement a polyfill for `Promise.race`

Problem Statement:

Suppose you are asked to implement your own version of `Promise.race`. The requirements are:

1. Input: an array of Promises (or values).
2. Output: a new Promise that settles as soon as **the first input settles** (resolve or reject).
3. Unlike `Promise.all`, we don't wait for everyone — only the first result matters.

For example:

```
const p1 = new Promise((res) => setTimeout(() => res("First"), 100));
const p2 = new Promise((res) => setTimeout(() => res("Second"), 200));

Promise.race([p1, p2]).then(console.log); // "First"
```

Intuition (what do we need?):

1. In `Promise.all`, we had to **collect results** and wait for all → but here we only care about the **first one to finish**.
2. That means no counters, no result arrays — just resolve/reject as soon as one settles.
3. But we must still handle **non-Promise values** gracefully → wrap with `Promise.resolve(p)`.
4. As soon as the first `.then` or `.catch` runs, we settle the outer promise.
5. Since a Promise can only settle once, later results are ignored automatically.

So the closure here just needs to **watch all promises** and settle at the first outcome.

Step-by-Step Plan:

- Create a wrapper promise → `new Promise((resolve, reject) => {...})`.
- Loop over the input array:
 - Wrap with `Promise.resolve(p)`.
 - Attach `.then(resolve).catch(reject)`.
- The first one to trigger wins.

Code:

```
function promiseRace(promises) {
  return new Promise((resolve, reject) => {
    // Edge case: empty input
    if (promises.length === 0) return;

    promises.forEach((p) => {
      Promise.resolve(p) // normalize to Promise
        .then(resolve)    // first resolve wins
        .catch(reject);   // first reject wins
    });
  });
}

// Example usage
const p1 = new Promise((res) => setTimeout(() => res("First"), 100));
const p2 = new Promise((res) => setTimeout(() => res("Second"), 200));
const p3 = new Promise((_ , rej) => setTimeout(() => rej("Error!"), 150));

promiseRace([p1, p2, p3])
  .then((val) => console.log("Resolved with:", val))
  .catch((err) => console.log("Rejected with:", err));
```

Step-by-Step Explanation:

1. Wrapper Promise

```
return new Promise((resolve, reject) => { ... });
```

- We create and return a new Promise.
- This is the promise that will behave like Promise.race.
- Its outcome (resolve or reject) depends on the **first promise in the input array that settles**

2. Iterating over input promises

```
promises.forEach((p) => { ... });
```

- The function accepts an array of promises (or even normal values).
- We loop through each one.
- **Promise.resolve(p)**

3. Promise.resolve(p)

Why? Because Promise.race must also work if the array contains non-Promise values.

Example:

```
Promise.race([42, Promise.resolve("Hi")])
```

- → should immediately resolve with 42.
- Wrapping each p with Promise.resolve ensures we always deal with a valid Promise.

4. Attaching .then and .catch

```
.then(resolve)
.catch(reject);
```

- For each promise:
 - If it resolves first → immediately call `resolve` of the wrapper promise.
 - If it rejects first → immediately call `reject` of the wrapper promise.
- Because the wrapper Promise **can only settle once**, whichever settles first “wins.”

Q16. Sequential vs Parallel Execution of Promises

Problem Statement:

Suppose you have an array of API functions that each return a Promise (e.g., fetching user data, posts, and comments).

- How would you run them **sequentially** (one after the other)?
- How would you run them **in parallel** (all at once)?

This is often asked to test whether you understand **Promise chaining vs Promise.all**.

Approach:

1. Parallel Execution

- Use `Promise.all` to fire all Promises together.
- Fastest, since they don't wait for each other.

2. Sequential Execution

- Use `reduce` or `for...of` with `await`.
- Each Promise starts **only after the previous one resolves**.

Code:

```
// Simulated async functions
function task(name, delay) {
  return new Promise((resolve) => {
    setTimeout(() => {
      console.log(`Finished ${name}`);
    }, delay);
    resolve();
  });
}
```



```

        resolve(name);
    }, delay);
});
}

const tasks = [
    () => task("Task 1", 1000),
    () => task("Task 2", 500),
    () => task("Task 3", 800)
];

// Parallel execution
async function runParallel() {
    console.log("Running in parallel:");
    await Promise.all(tasks.map((t) => t()));
    console.log("All done (parallel)");
}

// Sequential execution
async function runSequential() {
    console.log("Running sequentially:");
    for (let t of tasks) {
        await t();
    }
    console.log("All done (sequential)");
}

runParallel().then(() => runSequential());

```

Key Point for Interviews:

- **Parallel** → use `Promise.all` if tasks are independent (fastest).
- **Sequential** → use `await` in a loop if tasks depend on each other.

Q17. Implement a retry mechanism for a Promise

Problem Statement:

Suppose you are calling an API that sometimes fails due to network issues. You want to write a function `retry` that:

- Tries calling the function.
- If it fails, retries up to **N times** before finally rejecting.
- Each retry should happen after the previous one fails

Example:

```
retry(fetchData, 3).then(console.log).catch(console.error);
```

This should attempt `fetchData` up to 3 times before giving up.

Approach:

- Wrap the function in a loop of Promises using recursion or iteration.
- On failure (`catch`), decrement the attempts counter.
- If attempts remain → `retry`.
- If no attempts remain → `reject`.

```
function retry(fn, attempts) {  
  
  return new Promise((resolve, reject) => {  
  
    function attempt() {  
  
      fn()  
  
      .then(resolve)  
  
      .catch((err) => {  
  
        if (attempts-- > 0) {  
  
          console.log("Retrying...");  
  
          attempt(); // retry again  
  
        } else {
```

```

        reject(err);

    }

    });

}

attempt();

});

}

// Example usage

let count = 0;

function unreliableTask() {

    return new Promise((resolve, reject) => {

        count++;

        if (count < 3) reject("Failed attempt " + count);

        else resolve("Success on attempt " + count);

    });

}

retry(unreliableTask, 5)

    .then((res) => console.log(res))

    .catch((err) => console.error(err));

```

Expected Behavior:

- First two attempts fail → prints "Retrying..." each time.
- Third attempt succeeds → "Success on attempt 3" is resolved.
- If even after all retries it fails, the final rejection is returned.

Q18. What is prototypal inheritance in JavaScript?

Problem Statement:

In interviews, you might be asked: *How does JavaScript handle inheritance if it doesn't have classical classes like Java/C++?*

Solution:

- JavaScript uses **prototypal inheritance**.
- Every object has an internal link to another object called its **prototype**.
- If a property/method is not found on the object, JS looks up the prototype chain.

```
const parent = { greet: () => console.log("Hello from parent") };

const child = Object.create(parent);

child.sayHi = () => console.log("Hi from child");

child.sayHi();    // "Hi from child"

child.greet();    // "Hello from parent" (inherited)
```

Key Point:

- Objects can inherit directly from other objects.
- `Object.create(proto)` sets up the prototype chain.

- Modern `class` syntax is syntactic sugar over this mechanism.

Videos:

Prototypes - <https://www.scaler.com/topics/course/javascript-beginners/video/284/>

Inheritance - <https://www.scaler.com/topics/course/javascript-beginners/video/280/>

Q19. Difference between `__proto__`, `prototype`, and `constructor`

Problem Statement:

You may be asked: *Differentiate between `obj.__proto__`, `Function.prototype`, and `constructor`.*

Solution:

1. `__proto__`

- The actual prototype of an object (points to its parent).

```
const obj = {};  
  
console.log(obj.__proto__ === Object.prototype); // true
```

2. `prototype`

- A property of functions (especially constructor functions).
- Defines what properties new objects created with `new` will inherit.

```
function Person(name) { this.name = name; }
```

```
Person.prototype.sayHi = function() { console.log("Hi " + this.name); };

const p = new Person("Alice");

p.sayHi(); // "Hi Alice"
```

3. constructor

- Points back to the function that created the instance.

```
console.log(p.constructor === Person); // true
```

Key Point:

- `__proto__`: object's parent link.
- `prototype`: used to set inheritance when creating objects.
- `constructor`: reference to the function that created the object.

Q20. What happens when you use the new keyword in JavaScript?

Problem Statement:

An interviewer may ask: *Explain step by step what happens internally when you call a function with the new keyword.*

Solution:

Using new does four things:

1. Creates a **new empty object**.
2. Sets the object's prototype to the constructor function's `.prototype`.
3. Executes the constructor function with `this` bound to the new object.

4. Returns the new object (unless the constructor explicitly returns another object).

```
function Person(name) {  
  
  this.name = name;  
  
}  
  
Person.prototype.sayHi = function() {  
  
  console.log("Hi, " + this.name);  
  
};  
  
const p = new Person("Alice");  
  
p.sayHi(); // "Hi, Alice"
```

Key Point:

- new connects object instances with their prototype chain.
- Without new, the function would just run like a normal function.

Q21. Filter object's own properties and list keys, values, and entries

Problem Statement:

Suppose you have an object that inherits properties from its prototype. You are asked to:

1. Print only the object's **own properties** (not inherited ones).

2. Get its **keys, values, and entries** using built-in static methods.

For example:

```
const parent = { role: "admin" };

const child = Object.create(parent);

child.name = "Alice";

child.age = 25;
```

How would you extract only name and age (child's own properties) and ignore role?

Approach:

- Use `hasOwnProperty` to check if a property belongs directly to the object.
- Use `Object.keys`, `Object.values`, `Object.entries` for structured access.

Code:

```
const parent = { role: "admin" };

const child = Object.create(parent);

child.name = "Alice";

child.age = 25;

// Filtering own properties

for (let key in child) {

  if (child.hasOwnProperty(key)) {

    console.log("Own property:", key, "=", child[key]);

  }

}
```



```
// Using Object static methods

console.log(Object.keys(child)); // ["name", "age"]

console.log(Object.values(child)); // ["Alice", 25]

console.log(Object.entries(child));

// [["name", "Alice"], ["age", 25]]
```

Key Point:

- `for...in` iterates over all enumerable properties (own + inherited). Use `hasOwnProperty` to filter.
- `Object.keys` / `Object.values` / `Object.entries` return only **own enumerable properties** (ignore inherited ones automatically).

Q23. Difference between `call`, `apply`, and `bind`

Problem Statement:

In interviews, you might be asked: *How do `call`, `apply`, and `bind` work in JavaScript, and when would you use each?*

You are given an object and a standalone function, and you need to ensure the function uses the object's context (`this`).

Approach:

- `call`: invokes a function immediately, arguments passed individually.
- `apply`: invokes a function immediately, arguments passed as an array.
- `bind`: returns a new function with bound context, does not invoke immediately.

Code:

```
const person = { name: "Alice" };

function greet(greeting, punctuation) {

  console.log(greeting + " " + this.name + punctuation);

}

// call → arguments individually
greet.call(person, "Hello", "!");

// "Hello Alice!"

// apply → arguments as array
greet.apply(person, ["Hi", "!!"]);

// "Hi Alice!!"

// bind → returns a new function
const boundGreet = greet.bind(person, "Hey");

boundGreet("?");

// "Hey Alice?"
```

Key Point:

- Use `call` / `apply` when you want **immediate execution** with a specific `this`.

- Use bind when you want to create a new function with this permanently set.

Video:

Call, apply, bind -

<https://www.scaler.com/topics/course/javascript-beginners/video/285/>

Q24. Implement your own bind polyfill

Problem Statement:

Suppose you are asked to implement a polyfill for JavaScript's built-in bind method.

The polyfill should:

1. Return a new function with the given this context permanently bound.
2. Support **partial application** → arguments passed at binding time should be remembered.
3. Support **call-time arguments** → arguments passed when invoking the bound function should also work.

-

Example usage:

```
function greet(greeting, punctuation) {  
  
  console.log(greeting + " " + this.name + punctuation);  
  
}  
  
const person = { name: "Alice" };  
  
const boundGreet = greet.myBind(person, "Hello");  
  
boundGreet("!");
```

```
// Expected: "Hello Alice!"
```

Intuition (what do we need?):

1. When you call `greet.myBind(person, "Hello")`:
 - We don't want to **execute immediately** → just create a new wrapper function.
 - Later, when the wrapper runs, it should always use `person` as `this`.
2. We must remember two sets of arguments:
 - `args`: the ones passed during binding ("Hello").
 - `newArgs`: the ones passed when calling the bound function ("!").
 - Final arguments = `args.concat(newArgs)`.
3. Closures come into play:
 - We need to remember the original function (`fn`) and the pre-applied arguments (`args`).
4. Execution should be done with `apply`:
 - `fn.apply(context, [...args, ...newArgs])`.

Code:

```
Function.prototype.myBind = function(context, ...args) {  
  
  const fn = this; // the original function  
  
  return function(...newArgs) {  
  
    // When called, merge binding args + call-time args  
  
    return fn.apply(context, [...args, ...newArgs]);  
  
  };  
  
};
```

```
// Example usage

function greet(greeting, punctuation) {

  console.log(greeting + " " + this.name + punctuation);

}

const person = { name: "Alice" };

const boundGreet = greet.myBind(person, "Hello");

boundGreet("!"); // "Hello Alice!"
```

Key Point:

- myBind does not immediately call the function (unlike call/apply).
- It returns a new function with context fixed.
- Arguments can be partially applied.

Q25. Difference between Arrow Functions and Normal Functions in JavaScript

Problem Statement:

In interviews, you might be asked: *What's the difference between arrow functions and regular functions, especially in terms of this binding?*

Solution:

1. **this** behavior

- **Normal functions:** this is **dynamic** → depends on how the function is called.
- **Arrow functions:** this is **lexically bound** → taken from the surrounding scope where the function was defined.

Example:

```
const obj = {  
  
  name: "Alice",  
  
  normalFn: function() {  
  
    console.log("Normal:", this.name);  
  
  },  
  
  arrowFn: () => {  
  
    console.log("Arrow:", this.name);  
  
  }  
  
};  
  
obj.normalFn(); // "Normal: Alice"  
  
obj.arrowFn(); // "Arrow: undefined" (or global/outer scope)
```

2. Arguments object

- **Normal functions** have their own arguments object.
- **Arrow functions** do **not** have arguments – they use the outer scope's.

```
function normalFn(a, b) {  
  
  console.log(arguments); // [Arguments] { '0': 1, '1': 2 }
```

```

}

normalFn(1, 2);

const arrowFn = (a, b) => {

  console.log(arguments); // ReferenceError (no arguments object)

};

arrowFn(1, 2);

```

3. Constructor behavior

- **Normal functions** can be used as constructors with new.
- **Arrow functions** cannot (they throw an error).

```

function Person(name) { this.name = name; }

const p = new Person("Alice"); // Works

const ArrowPerson = (name) => { this.name = name; };

// const ap = new ArrowPerson("Bob"); // Error: ArrowPerson is not a constructor

```

Key Point:

- Use **arrow functions** for callbacks and preserving lexical this.
- Use **normal functions** when you need this, arguments, or constructor behavior.

Video:

This keyword - <https://www.scaler.com/topics/course/javascript-beginners/video/274/>

Q26. Fix the this issue in setTimeout

Problem Statement:

Suppose you have an object with a method that logs its name after a delay. Using `setTimeout` with a normal function often causes the `this` context to be lost, because `setTimeout` executes the callback in the global scope.

```
const person = {  
  
  name: "Alice",  
  
  greet: function() {  
  
    setTimeout(function() {  
  
      console.log("Hello, " + this.name);  
  
    }, 1000);  
  
  }  
};  
  
person.greet();  
  
// Output after 1s: "Hello, undefined" (this is lost)
```

How would you fix this so that the correct `this.name` is used?

Approach:

- Problem: Normal function in `setTimeout` has `this` pointing to window (or undefined in strict mode).
- Solutions:
 1. Use an **arrow function** (inherits lexical `this`).
 2. Or capture `this` in a variable (`const self = this`).
 3. Or use `.bind(this)`.

```
const person = {  
  
  name: "Alice",  
  
  greet: function() {  
  
    setTimeout(() => {  
  
      console.log("Hello, " + this.name);  
  
    }, 1000);  
  
  }  
};  
  
person.greet();  
  
// Output: "Hello, Alice"
```

Alternative Fixes:

1. Using `.bind(this)`

```
setTimeout(function() {  
  
  console.log("Hello, " + this.name);  
  
}, 1000);
```

```
}.bind(this), 1000);
```

2. Using `self = this`

```
const self = this;

setTimeout(function() {

  console.log("Hello, " + self.name);

}, 1000);
```

Key Point:

- Arrow functions don't have their own `this`; they inherit it from the enclosing scope.
- This makes them ideal for callbacks like `setTimeout` or event handlers where you want to preserve context.

Q27. What is the difference between shallow copy and deep copy in JavaScript?

Problem Statement:

In interviews, you may be asked: *What's the difference between shallow and deep copy of an object, and how do you implement each?*

Solution:

1. Shallow Copy

- Copies only the first level.
- Nested objects/arrays still refer to the same memory (shared).

```
const obj = { name: "Alice", address: { city: "Paris" } };

const shallow = { ...obj };

shallow.address.city = "London";

console.log(obj.address.city); // "London" (changed in both)
```

Ways to do shallow copy:

- Spread operator { ...obj }
- Object.assign({}, obj)

2. Deep Copy

- Copies all levels, creating **independent clones**.
- Changing nested objects does not affect the original.

```
const obj = { name: "Alice", address: { city: "Paris" } };

// Deep copy using JSON

const deep = JSON.parse(JSON.stringify(obj));

deep.address.city = "London";

console.log(obj.address.city); // "Paris" (unchanged)
```

Other ways to deep copy:

- `structuredClone(obj)` (modern browsers).
- Using libraries like Lodash (`_ .cloneDeep`).

Key Point:

- **Shallow copy** → fast, but nested references still linked.
- **Deep copy** → safe for nested structures, but slower and may lose special object types (like `Date`, `Map`) if using `JSON.parse/stringify`.

Videos:

Shallow copy - <https://www.scaler.com/topics/course/javascript-beginners/video/300/>

Deep Copy - <https://www.scaler.com/topics/course/javascript-beginners/video/301/>

Q28. Implement a deep clone function without using JSON

Problem Statement:

You are asked to implement a function `deepClone` that creates a true deep copy of an object.

Constraints:

1. It should handle nested objects and arrays.
2. It should **not** rely on `JSON.parse(JSON.stringify())` (because that fails for `Date`, `Map`, `Set`, functions, etc.).
3. Modified clone should not affect the original.

```
const person = {  
  name: "Ravi",  
  details: { city: "Delhi", hobbies: ["cricket", "music"] }  
};
```

```
const clone = deepClone(person);
clone.details.city = "Mumbai";

console.log(person.details.city); // "Delhi" (original unchanged)
console.log(clone.details.city);  // "Mumbai"
```

Intuition (what do we need?):

1. **Primitives** (number, string, boolean, null, undefined) → just return them.
2. **Date objects** → create a new Date instance.
3. **Arrays** → create a new array and recursively deep clone each element.
4. **Objects** → create a new object and recursively clone each property.
5. Special case: null → though typeof is "object", it should return null.

Whenever you hear “remember structure and dive inside deeply” → **recursion** is the natural fit.

Step-by-Step Plan:

- Base case: if not an object → return directly.
- Handle specific cases (Date, Array).
- Otherwise, assume it's an object → create {} and loop through keys.
- Use `hasOwnProperty` to skip inherited properties.
- Recursively call `deepClone` on nested values.

```
function deepClone(value) {
  // Step 1: handle primitives and null
  if (value === null || typeof value !== "object") {
    return value;
  }

  // Step 2: handle Date
  if (value instanceof Date) {
    return new Date(value);
  }
```

```

}

// Step 3: handle Array
if (Array.isArray(value)) {
  return value.map(item => deepClone(item));
}

// Step 4: handle Object
const clonedObj = {};
for (let key in value) {
  if (value.hasOwnProperty(key)) {
    clonedObj[key] = deepClone(value[key]);
  }
}
return clonedObj;
}

// Example usage
const person = {
  name: "Ravi",
  details: { city: "Delhi", hobbies: ["cricket", "music"] },
  dob: new Date("1995-06-15")
};

const clone = deepClone(person);

// Modify clone
clone.details.city = "Mumbai";
clone.details.hobbies[0] = "football";

console.log(person.details.city);    // "Delhi"
console.log(person.details.hobbies); // ["cricket", "music"]
console.log(clone.details.city);     // "Mumbai"
console.log(clone.dob instanceof Date); // true

```

Expected Behavior:

- clone is completely independent from obj.

- Nested modifications in `clone` don't affect `obj`.
- Special objects like `Date` are preserved.

Key Point:

- This implementation covers primitives, arrays, plain objects, and dates.
- For full robustness (like handling `Map`, `Set`, `RegExp`, circular references), we'd need a more advanced version (like `structuredClone` or libraries such as `Lodash`).

Q29. Implement a polyfill for `Array.prototype.reduce`

Problem Statement:

In interviews, you may be asked: *How does `reduce` work internally? Can you implement your own version?*

The `reduce` method executes a reducer function on each element of the array, resulting in a single accumulated output.

Example:

```
[1, 2, 3, 4].reduce((acc, curr) => acc + curr, 0);
```

```
// Output: 10
```

We need to implement `myReduce` that behaves the same way.

Approach:

- Extend `Array.prototype` with `myReduce`.
- Take a callback and an optional initial value.
- If no initial value, use the first element as the accumulator.

- Iterate over the array, calling the callback on each element.

Code:

```
Array.prototype.myReduce = function(callback, initialValue) {

  if (typeof callback !== "function") {

    throw new TypeError(callback + " is not a function");

  }

  let accumulator = initialValue;

  let startIndex = 0;

  // If no initialValue, take first element as accumulator

  if (accumulator === undefined) {

    if (this.length === 0) {

      throw new TypeError("Reduce of empty array with no initial value");

    }

    accumulator = this[0];

    startIndex = 1;

  }

  for (let i = startIndex; i < this.length; i++) {

    accumulator = callback(accumulator, this[i], i, this);

  }

}
```



```
return accumulator;

};

// Example usage

const sum = [1, 2, 3, 4].myReduce((acc, curr) => acc + curr, 0);

console.log(sum); // 10

const product = [1, 2, 3, 4].myReduce((acc, curr) => acc * curr);

console.log(product); // 24
```

Video:

Reduce method -

<https://www.scaler.com/topics/course/javascript-beginners/video/271/>

Q30. Implement debounce(fn, delay)

Problem Statement:

You're building a search box that fires an API call on every keypress. This is wasteful. Implement a debounce utility so that fn runs only **after the user stops typing for delay ms**.

Scenario:

You're typing in a search box. Each keystroke fires an event. If you hit 10 keys in 2 seconds, that's 10 API calls — wasteful!

Instead, we want: **Only fire after the user pauses for X ms.**

Intuition (what do we need?):

1. We need to wait for a quiet period before firing.
2. If another call happens before the wait finishes → cancel the old wait and start a new one.
3. That means we must remember the timer ID across calls.
4. When you think “remember across calls” → closure is the natural fit.

Implementation Plan (step by step):

- Keep a variable `timerId` in the closure.
- Return a function (the one user actually calls).
- Every time it's called:
 - Cancel the old timer (`clearTimeout(timerId)`).
 - Start a new one with `setTimeout`.
 - After delay, finally execute the original function with latest arguments.

```
function debounce(fn, delay) {  
  let timerId; // remembered across calls (closure)  
  
  return function(...args) { // inner function (user calls this)  
    const context = this;  
  
    // Step 1: cancel any previously scheduled execution  
    clearTimeout(timerId);  
  
    // Step 2: start a new timer  
    timerId = setTimeout(() => {  
      fn.apply(context, args); // run fn with latest args  
    }, delay);  
  };  
}
```

Expected Behavior:

- Rapid calls reset the timer; only the **last** call's args are used.

- Useful for input handlers, window resize, auto-save, etc.

Q31. Implement `throttle(fn, interval)`

Problem Statement:

You're handling a scroll event and want to limit the callback to at most **once every `interval` ms**, regardless of how many times the event fires in between.

Scenario:

You're scrolling. The scroll event fires **dozens of times per second**. If your handler is heavy (like image loading), it can choke the browser.

Instead, we want: **Fire at most once every `X` ms.**

Intuition (what do we need?):

1. We need to limit calls to fixed intervals.
2. That means: keep track of when the function last ran.
3. If another call comes before interval ends, we ignore it.
4. But sometimes we might want to capture the latest arguments so when the interval ends, we fire one last call.
5. So we need to remember two things in closure:
 - `last` (timestamp of last call) or `timerId`.
 - `lastArgs` (to remember latest arguments).

Approach:

- Track the last execution time (timestamp) in a closure.
- If enough time has passed since the last call, run `fn`; otherwise ignore (or queue).

Code (timestamp-based throttle):

```
function throttle(fn, interval) {

  let timerId = null;    // to block execution while timer runs

  let lastArgs = null;   // to remember latest arguments


  return function(...args) {

    const context = this;

    lastArgs = args;


    if (!timerId) {

      // Step 1: run immediately (leading call)

      fn.apply(context, lastArgs);


      // Step 2: start timer

      timerId = setTimeout(() => {

        timerId = null;


        // Step 3: if new args arrived, run trailing call

        if (lastArgs) {

          fn.apply(context, lastArgs);

          lastArgs = null;

        }

      }, interval);

    }

  }

}
```

```
};  
  
}
```

Expected Behavior:

- During a burst, fn executes immediately, then **suppresses** calls until interval passes, then allows the next one.
- Great for scroll, resize, mousemove where you want a steady cadence.

Alternative (timer-based throttle with trailing call):

```
function throttleTrailing(fn, interval) {  
  
  let timerId = null;  
  
  let lastArgs = null;  
  
  return function(...args) {  
  
    lastArgs = args;  
  
    const context = this;  
  
    if (timerId === null) {  
  
      fn.apply(context, lastArgs); // leading call  
  
      timerId = setTimeout(() => {  
  
        timerId = null;  
  
        // Optional: run trailing call if new args arrived during the wait  
  
        if (lastArgs !== null) {  
  
          fn.apply(context, lastArgs);  
  
        }  
  
      }, interval);  
  
    }  
  
  }  
}
```

```
        lastArgs = null;

    }

    }, interval);

}

};

}
```

Key Differences (Debounce vs Throttle):

- **Debounce:** waits for quiet time; emits once after inactivity.
- **Throttle:** emits at a fixed max rate; ignores intermediate calls.

Q32. Implement a function to flatten a nested array

Problem Statement:

You're given a deeply nested array, e.g.:

```
const arr = [1, [2, [3, [4]], 5]];
```

You are asked to implement a function `flatten` that transforms it into:

```
[1, 2, 3, 4, 5]
```

Constraints:

- Must handle arrays nested at any depth.
- Avoid using `Array.prototype.flat(Infinity)` directly (since interviewer wants to test recursion/iteration skills).

Intuition (what do we need?):

1. Flattening means:
 - If element is a primitive → push directly into result.
 - If element is an array → recursively break it down until primitives.
2. Whenever you see “process structure inside structure until no more” → **recursion** is a natural fit.
3. Alternatively, you can solve with iteration + stack, but recursion is the clearest first solution.

Step-by-Step Plan:

- Create an empty result array.
- Loop through each element in input array.
- If it's an array → recursively flatten it and concat into result.
- If not → push directly into result.
- Return result.

Code (recursive solution):

```
function flatten(arr) {  
  let result = [];  
  
  for (let item of arr) {  
    if (Array.isArray(item)) {  
      result = result.concat(flatten(item)); // recurse  
    } else {  
      result.push(item);  
    }  
  }  
  
  return result;  
}  
  
// Example usage  
const arr = [1, [2, [3, [4]], 5]];  
console.log(flatten(arr));  
// [1, 2, 3, 4, 5]
```

Alternative (using reduce):

```
function flatten(arr) {  
  return arr.reduce((acc, item) => {  
    if (Array.isArray(item)) {  
      return acc.concat(flatten(item));  
    } else {  
      return acc.concat(item);  
    }  
  }, []);  
}
```

Q33. Implement currying for infinite sum

Problem Statement:

You are asked to implement a function sum that can be called in a chain like this:

```
console.log(sum(1)(2)(3)(4)()); // 10  
console.log(sum(5)(10)(15)()); // 30
```

This requires:

- Supporting indefinite chaining of function calls.
- Returning the accumulated sum when the chain ends.

Code :

```
function sum(x) {  
  
  let total = x;  
  
  function inner(y) {  
  
    if (y !== undefined) {  
  
      total += y;  

```



```

    return inner; // allow chaining

  }

  return total; // stop condition
}

return inner;
}

// Example usage

console.log(sum(1) (2) (3) (4) ()); // 10

console.log(sum(5) (10) (15) ()); // 30

```

Q34. Implement a custom Event Emitter

Problem Statement:

You're asked to implement an `EventEmitter` class that behaves like Node.js's `EventEmitter`.

It should:

1. Allow registering event listeners with `.on(event, callback)`.
2. Allow removing listeners with `.off(event, callback)`.
3. Allow triggering all listeners with `.emit(event, ...args)`.

```
const emitter = new EventEmitter();
```

```
function greet(name) {

  console.log("Hello, " + name);

}

emitter.on("greet", greet);

emitter.emit("greet", "Ravi"); // "Hello, Ravi"

emitter.off("greet", greet);

emitter.emit("greet", "Ravi"); // (nothing happens)
```

Intuition (what do we need?):

1. We need a way to **store events and their listeners** → use an object like:

```
{

  greet: [callback1, callback2],

  login: [callback3]

}
```

1. `.on(event, fn)` → push listener into that event's array.
2. `.emit(event, args)` → loop through listeners and call them with args.
3. `.off(event, fn)` → remove the specific callback from the event's array.

Whenever you think “we need to remember sets of things and update them later” → **closure inside a class/object** is the right fit.

Step-by-Step Plan:

- Create a class EventEmitter.
- Maintain an internal events map (event → listeners).
- .on: add callback to event's array.
- .emit: loop and call all callbacks with arguments.
- .off: filter out the callback from the event's array.

Code:

```
class EventEmitter {

  constructor() {

    this.events = {}; // event → [listeners]

  }

  on(event, listener) {

    if (!this.events[event]) {

      this.events[event] = [];

    }

    this.events[event].push(listener);

  }

  off(event, listener) {

    if (!this.events[event]) return;

  }

}
```

```
    this.events[event] = this.events[event].filter(  
        (l) => l !== listener  
    );  
}
```

```
emit(event, ...args) {  
    if (!this.events[event]) return;  
  
    this.events[event].forEach((listener) => {  
        listener.apply(this, args);  
    });  
}
```

```
// Example usage
```

```
const emitter = new EventEmitter();
```

```
function greet(name) {  
    console.log("Hello, " + name);  
}
```

```
emitter.on("greet", greet);
```

```
emitter.emit("greet", "Ravi"); // "Hello, Ravi"
```

```
emitter.off("greet", greet);  
  
emitter.emit("greet", "Ravi"); // nothing
```

Expected behavior:

```
emitter.on("event1", () => console.log("First listener"));  
  
emitter.on("event1", () => console.log("Second listener"));  
  
emitter.emit("event1");  
  
// "First listener"  
  
// "Second listener"
```

Q36. Difference between async, defer, and normal <script> loading

- **Normal script** → blocks HTML parsing until the script loads & executes.
- **async** → script loads asynchronously, executes immediately when ready (order not guaranteed).
- **defer** → script loads asynchronously, but executes after HTML parsing (order preserved).

Expected Interview Trick: *“Which one would you use for analytics scripts vs app code?”*

- Use async for independent scripts (ads, analytics).
- Use defer for app scripts that need DOM ready.

Q36. Difference between async, defer, and normal <script> loading

Problem Statement:

In interviews, you may be asked: *“What is the difference between a normal <script> tag, one with async, and one with defer? How do they affect page rendering and performance?”*

Intuition (what do we need to understand?):

1. The browser parses HTML **line by line** (top → bottom).
2. When it encounters a <script> tag, what happens?
 - Normally → parsing **stops**, script loads, executes, then parsing resumes.
 - With async → script loads in parallel and executes **as soon as it's ready** (parsing may pause mid-way).
 - With defer → script loads in parallel but executes **only after HTML is fully parsed**.

So the core is about **when does the script block HTML parsing and when does it execute**.

Step-by-Step Behavior

1. Normal <script>

```
<script src="main.js"></script>
```

- Browser stops parsing HTML.
- Loads the script → executes it immediately.
- Then continues parsing.

2. Async <script>

```
<script src="analytics.js" async></script>
```

- Script loads in parallel with HTML parsing.
- Executes **immediately when ready**, pausing HTML parsing if needed.
- Order of execution **not guaranteed** (whichever finishes first runs first).

Good for independent scripts (analytics, ads).

Dangerous if script depends on DOM or other scripts.

3. Defer <script>

```
<script src="app.js" defer></script>
```

- Script loads in parallel with HTML parsing.
- Execution delayed until **HTML parsing is complete**.
- Scripts with defer execute in the order they appear.

Best for app scripts that depend on DOM being ready.

Suppose your HTML has:

```
<head>

<script src="a.js"></script>

<script src="b.js" async></script>

<script src="c.js" defer></script>

</head>
```

- `a.js` → blocks immediately, runs before parsing continues.
- `b.js` → loads async, may run anytime when ready (maybe before or after parsing ends).
- `c.js` → loads async but guaranteed to run **after parsing ends** and in order.

Key Points (Interview Takeaways):

- **Normal**: blocks HTML parsing → bad for performance.
- **Async**: non-blocking, executes ASAP, order not guaranteed → good for independent scripts.
- **Defer**: non-blocking, executes after parsing, order preserved → best for structured app scripts.
- Always prefer `defer` for main application scripts.
- Use `async` for analytics/ads/third-party scripts.

Q37. Explain the Critical Rendering Path (CRP)

Problem Statement:

In interviews, you may be asked:

“What happens inside the browser from the moment HTML arrives until the page is painted? Can you explain the Critical Rendering Path and optimizations?”

This is a **system-level question** that tests whether you understand how JS, CSS, and HTML interact to produce pixels on screen.

Intuition (what do we need to know?):

1. The browser doesn't directly show raw HTML text → it must **parse, compute, and render**.
2. Each resource (HTML, CSS, JS) can either **help** or **block** this rendering.

3. Understanding CRP means knowing the **steps the browser takes** before pixels show up.

Step-by-Step Process (CRP Stages):

1. HTML → DOM (Document Object Model)

- Browser parses HTML → creates a tree of elements.
- Example:

```
<body>

  <h1>Hello</h1>

</body>
```

- → DOM Tree: Document → body → h1 → text.

2. CSS → CSSOM (CSS Object Model)

- Browser parses CSS → builds a tree of style rules.

3. DOM + CSSOM → Render Tree

- Browser combines structure (DOM) and style (CSSOM).
- Invisible elements (`display: none`) are skipped.

4. Layout (Reflow)

- Browser calculates position and size of each element (box model).
- Example: `h1` → (x=0, y=0, width=500px, height=50px).

5. Paint

- Browser fills pixels (colors, borders, text, images).

6. Composite

- Layers (e.g., for CSS transforms, z-index) are drawn together into the final screen image.

Visualization of CRP

HTML → DOM

CSS → CSSOM

DOM + CSSOM → Render Tree

Render Tree → Layout → Paint → Composite

Where is JavaScript in this?

- **Blocking scripts** (<script>) pause HTML parsing.
- **Blocking CSS** (stylesheets) pause JS execution until loaded (because JS may query styles).
- This is why script placement and attributes (async, defer) matter so much.

Optimizations (Interview Gold):

1. **Minimize render-blocking resources**
 - Use async / defer for scripts.
 - Inline critical CSS, defer non-critical CSS.
2. **Reduce reflows/repaints**
 - Batch DOM changes.
 - Avoid inline style changes in loops.
3. **Optimize images**
 - Use responsive, compressed images.
 - Lazy load below-the-fold images.
4. **Measure with DevTools**

- Performance tab → see layout, paint, composite breakdown.

Key Points:

- CRP = sequence browser takes to render pixels.
- Steps: **DOM** → **CSSOM** → **Render Tree** → **Layout** → **Paint** → **Composite**.
- JavaScript & CSS can **block rendering**.
- Optimizations: async/defer, critical CSS, minimize reflows, lazy load assets.

Q38. What are Web Vitals (LCP, FID, CLS) and how do you optimize them?

Problem Statement:

In interviews, you may be asked:

“What are Core Web Vitals, why are they important, and how can you improve them?”

This is a **modern frontend question** that checks your knowledge of performance and user experience metrics.

Intuition (what do we need to know?):

1. Web Vitals are **metrics that measure user experience** — not just technical load times.
2. Google defined **three core metrics** (LCP, FID, CLS) to reflect **loading speed, interactivity, and visual stability**.
3. These directly affect SEO rankings and user perception of app quality.

Step-by-Step

1. LCP (Largest Contentful Paint) – Loading Performance

- Definition: Time taken to render the **largest visible element** (text block, image, video)

- Good: < **2.5s** | Needs Improvement: 2.5–4s | Poor: > 4s.
- Example:
 - If your hero image takes 5s → bad LCP.
 - If your text loads instantly but big banner image is slow → bad LCP.

Optimizations:

- Compress and serve images in modern formats (WebP, AVIF).
- Use a CDN for static assets.
- Preload critical fonts/images.
- Remove render-blocking JS/CSS.

2. FID (First Input Delay) – Interactivity

- Definition: Delay between user's first interaction (click, keypress) and browser responding.
- Good: < **100ms** | Needs Improvement: 100–300ms | Poor: > 300ms.
- Example:
 - If user clicks a button but JS is still executing a heavy loop for 2s → high FID.

Optimizations:

- Break up long JavaScript tasks (>50ms) using `requestIdleCallback` or `setTimeout`.
- Use **code-splitting** (load only necessary JS upfront).
- Defer non-critical scripts.
- Optimize event listeners.

3. CLS (Cumulative Layout Shift) – Visual Stability

- Definition: Measures unexpected visual shifts (elements moving without user interaction).
- Score: < **0.1** is Good, 0.1–0.25 Needs Improvement, >0.25 Poor.
- Example:

- Page loads, text is visible, but suddenly an ad pushes it down → bad CLS.

Optimizations:

- Always set width/height on images and ads.
- Use `font-display: swap` for fonts.
- Avoid inserting DOM elements above existing content dynamically.

Story / Analogy

- Imagine you're in a restaurant:
 - **LCP** = how fast the waiter brings your main dish.
 - **FID** = how fast the waiter responds when you call him.
 - **CLS** = whether your table suddenly shifts while eating.

All three matter for a good dining (or browsing) experience.

Key Points (Interview Takeaways):

- **LCP** → Measures **loading performance**. Target < 2.5s.
- **FID** → Measures **interactivity**. Target < 100ms.
- **CLS** → Measures **visual stability**. Target < 0.1.
- Tools to measure: Lighthouse, Chrome DevTools, PageSpeed Insights.
- Optimizations involve **faster resource loading, smaller JS, stable layouts**.

Q39. How does JavaScript garbage collection work?

Problem Statement:

In interviews, you may be asked:

"How does memory management work in JavaScript? What is garbage collection, and how does the engine decide what memory to free?"

This is a **deep internals question** that tests if you understand how JS handles memory automatically and why memory leaks can still happen.

Intuition (what do we need to know?):

1. JavaScript manages memory automatically (no malloc/free like C).
2. But memory isn't infinite — browser must **free objects that are no longer needed**.
3. Core question: *"How does JS know an object is no longer needed?"*
→ Answer: **Reachability**.

Step-by-Step

1. Memory Lifecycle in JS

- Allocate → Use → Release.
- Example:

```
let user = { name: "Ravi" }; // allocate
```

```
console.log(user.name);    // use
```

```
user = null;               // release (eligible for GC)
```

Reachability Concept

- An object is considered "alive" if it's reachable from **roots**.
- **Roots**: global object (window in browsers, global in Node), current execution stack, and variables in closures.
- If an object can't be reached → marked for deletion.

3. Mark-and-Sweep Algorithm (simplified)

- Step 1: Start at roots, "mark" all reachable objects.
- Step 2: Sweep through heap, remove unmarked (unreachable) ones.
- This runs automatically, periodically.

4. Why memory leaks still happen

Even with GC, leaks happen when references are kept alive unintentionally:

- **Closures holding references**

```
function leaky() {  
  
  let bigArray = new Array(1000000).fill("data");  
  
  return function() {  
  
    console.log(bigArray.length); // closure keeps bigArray alive  
  
  };  
  
}  
  
const leak = leaky();  
  
// bigArray stays in memory forever!
```

Detached DOM nodes

```
let el = document.getElementById("myDiv");  
  
document.body.removeChild(el);  
  
// if still referenced in JS → not garbage collected
```

- **Event listeners not removed** → DOM nodes stay alive.

Q40. Event Bubbling vs Capturing (and Event Delegation)

Problem Statement:

In interviews, you may be asked:

“How does event propagation work in the DOM? What’s the difference between bubbling and capturing? How can event delegation help performance?”

This checks both your **JS fundamentals** and **frontend optimization knowledge**.

Intuition (what do we need to know?):

1. When you click an element inside nested elements, the event doesn’t just stay there.
2. It travels through the DOM in **phases**.
3. Two main phases matter:
 - **Capturing (trickling down)** → root → parent → child.
 - **Bubbling (bubbling up)** → child → parent → root.

Default in browsers = **bubbling phase**.

Step-by-Step Propagation

Suppose we have:

```
<div id="parent">  
  
  <button id="child">Click Me</button>  
  
</div>
```

1. Capturing phase

- Event goes from document → <html> → <body> → <div> → <button>.

2. Target phase

- Event reaches the actual target (<button>).

3. Bubbling phase

- Event goes back up: <button> → <div> → <body> → <html> → document.

Event Delegation (Optimization)

- Instead of attaching a listener to every child, attach **one listener on the parent**.
- Use `event.target` to figure out which child was clicked.

Example:

```
document.getElementById("parent").addEventListener("click", (e) => {  
  
  if (e.target.tagName === "BUTTON") {  
  
    console.log("Button clicked:", e.target.id);  
  
  }  
  
});
```

- Now even if you dynamically add 100 buttons inside parent, you don't need new listeners.
 - Saves memory.
 - Reduces performance overhead.
 - Works great with dynamically generated DOM.

Q41. How does the JavaScript engine execute code?

Problem Statement:

Interviewers often ask:

"Can you explain how JavaScript executes code under the hood? What are the call stack, heap, event loop, microtasks, and macrotasks?"

This checks whether you understand JS execution internals, async behavior, and why promises/setTimeout behave differently.

Intuition (what do we need to know?):

1. JavaScript is **single-threaded** → one thing at a time.
2. To handle async tasks, it uses a **call stack + memory heap + event loop + task queues**.
3. Execution is like a play:
 - **Heap** = backstage storage (objects, data).
 - **Call Stack** = stage (functions run here, one at a time).
 - **Event Loop** = director, deciding when the next actor (task) comes on stage.
 - **Task Queues** = actors waiting backstage (macrotasks & microtasks).

Step-by-Step

1. Heap

- Memory storage for objects, arrays, functions.
- Example:

```
let user = { name: "Rohit" }; // stored in heap
```

2. Call Stack

- Executes functions in LIFO (Last In First Out).
- Example:

```
function a() { b(); }
```

```
function b() { console.log("Hi"); }
```

```
a();
```

Call stack evolution:

```
a()
```

```
b()
```

console.log()

3. Web APIs (Browser APIs)

- For async tasks like setTimeout, DOM events, fetch.
- These don't run in the call stack; they're handled by browser runtime.

4. Task Queues

- Once async work is ready, callbacks go into queues.
- Two types:
 - **Macrotask Queue** → setTimeout, setInterval, DOM events, I/O.
 - **Microtask Queue** → Promise.then, queueMicrotask, MutationObserver.

5. Event Loop

- Constantly checks:
 - Is the **call stack empty**?
 - If yes → take next task from **microtask queue first**, then macrotask queue.
- Microtasks always run before macrotasks.

```
console.log("Start");

setTimeout(() => console.log("Timeout"), 0);

Promise.resolve().then(() => console.log("Promise"));

console.log("End");
```

Execution order:

1. "Start" → call stack.
2. setTimeout → goes to Web API → macrotask queue.
3. Promise.then → goes to microtask queue.
4. "End" → call stack.
5. Event loop: stack empty → run microtasks first → "Promise".
6. Then macrotasks → "Timeout".

Output:

Start

End

Promise

Timeout